Presentation script

Three presenters / 11 slides / target duration 7 minutes 30 seconds

This script follows the improved PowerPoint. Assign any three team members to the presenter labels. Read the spoken paragraphs, not the slide headings or timing labels.

Presenter	Slides	Time window
Presenter 1	1-4	0:00-2:30
Presenter 2	5-7	2:30-5:05
Presenter 3	8-11	5:05-7:30

How to stay below eight minutes

Use the cumulative timestamps as rehearsal checkpoints. Keep the two handovers short. Offer a live demonstration after the timed presentation rather than opening services during the talk. The ending includes a five-second buffer.

A time estimate is a rehearsal target. Technical terms and large numbers take longer to say than ordinary words. Do one complete practice with actual slide changes and adjust your pace before presenting.

Pronunciation reminders

Term	Say it as
ETL	E-T-L
SQL	S-Q-L or sequel, consistently
CRUD	crud
OLAP	oh-lap
API	A-P-I

Simple meanings if a question comes up

Transaction: database changes commit together or roll back. **Idempotent insertion:** retrying the same event ID does not add another valid row. **Staging:** a waiting area before trusted historical data.

Presenter 1

Slides 1-4 / cumulative time 0:00-2:30

Slide 1 - Introduction (0:00-0:20)

Good morning. Our project is PitchPulse360, a football analytics and data engineering platform. It processes historical match data across five European leagues and six seasons. We built a complete workflow that collects the data, validates it, stores it in different systems and presents the results through a dashboard and API.

Slide 2 - Project workflow (0:20-0:55)

The platform has two processing paths. The first is a batch pipeline for historical CSV files. It extracts records, cleans them, applies a quality gate and prepares the databases and analytical tables. The second handles individual match events through FastAPI and RabbitMQ, storing accepted events in PostgreSQL staging. Docker Compose runs the supporting services together. Prefect coordinates the batch pipeline. These paths remain separate: receiving an event does not automatically change the historical league standings.

Slide 3 - Cleaning and the quality gate (0:55-1:40)

The extractor reads 30 league-season files and standardizes their columns. Cleaning converts dates and numeric types, normalizes selected team names and checks duplicate fixtures and impossible measurements. A real example is Newcastle versus West Ham. The source records eight away shots but nine shots on target. We preserve the valid score and mark both inconsistent shot measurements as unknown. After cleaning, a custom Pandas expectation suite checks required fields, uniqueness, ranges and football consistency. All 45 expectations pass on the clean data. A separate damaged sample triggers nine failed expectations. If the clean dataset fails this gate, warehouse loading stops.

Slide 4 - Relational storage and ETL (1:40-2:25)

PostgreSQL stores the core data in a normalized relational model. Separate tables represent countries, leagues, seasons, competition editions, teams and matches. Primary and foreign keys maintain relationships. Unique and check constraints prevent repeated fixtures and invalid values. For loading, PostgreSQL COPY transfers cleaned records into a temporary table. A transaction then refreshes the historical snapshot and commits the changes together. Stable fixture IDs and uniqueness rules make reruns safe without duplicating matches. The verified warehouse contains 10,781 matches and 30,186 goals. A separate analytical table partitions records by season.

Handover (2:25-2:30)

I'll now hand over to explain the analytical queries and additional storage and ingestion features.

Presenter 2

Slides 5-7 / cumulative time 2:30-5:05

Slide 5 - Reusable SQL and football summaries (2:30-3:20)

We use SQL views to define calculations once and reuse them consistently. Football views calculate daily goals, team win percentages and season rankings. Window functions include DENSE_RANK for ranking, LAG and LEAD for season comparisons, and running SUM for cumulative points. For the required business metrics, we generate 6,000 synthetic ticket orders. These support daily revenue, monthly growth and repeat-customer percentage. They do not represent actual club finances. For multidimensional analysis, ROLLUP produces hierarchical subtotals, CUBE includes every grouping combination, and GROUPING SETS selects specific combinations. We execute these analytical queries in PostgreSQL and DuckDB and verify that their results match.

Slide 6 - Delta history and local processing (3:20-4:10)

The lakehouse organizes data into Bronze, Silver and Gold layers. Bronze preserves selected raw records, Silver contains cleaned matches, and Gold contains team-season summaries. A separate Delta experiment creates versions through insertion, updating and deletion. We then read version zero to demonstrate time travel. Two incompatible writes fail without creating another committed version. For the performance experiment, we repeat historical records to create 250,000 benchmark rows. DuckDB queries identical data stored as CSV, Parquet and native database tables, using one-thread and four-thread limits. Every mode receives a warm-up and five timed runs. Answers must match before we interpret the timings.

Slide 7 - Documents and incoming match events (4:10-5:00)

MongoDB preserves 10,781 raw match documents. A separate scout-report collection demonstrates insertion, filtered reads, updates and deletion, together with compound indexes. For incoming events, FastAPI validates the JSON payload and publishes valid messages to RabbitMQ. The API returns acceptance after the broker confirms publication. The consumer validates each message again and inserts it into PostgreSQL staging. It acknowledges the message only after the database transaction commits. Database failures cause retries, while a unique event ID prevents duplicate valid staging rows. This provides at-least-once delivery with idempotent insertion. Invalid messages receive rejection records and move to a separate rejected queue.

Handover (5:00-5:05)

The final section covers how we automate the platform and demonstrate its results.

Presenter 3

Slides 8-11 / cumulative time 5:05-7:30

Slide 8 - Automation and verification (5:05-5:50)

Prefect orchestrates eleven dependent tasks, beginning with extraction and ending with streaming verification. The quality gate runs before warehouse loading. Later tasks prepare metric views, synchronize MongoDB, compare analytical queries and execute the lakehouse and benchmark demonstrations. Task retries handle temporary failures. A deployment concurrency limit and shared filesystem lock prevent overlapping batches from replacing the same outputs. The default schedule runs every 3,600 seconds while Docker and the services remain active. Verification checks actual database results and Prefect execution states. The recorded completed flow contains all eleven successful tasks.

Slide 9 - The football dashboard (5:50-6:35)

The Streamlit dashboard reads prepared Silver and Gold lakehouse tables, with DuckDB supporting analytical queries. Users select a league and season to explore standings, points distributions, attacking and defensive comparisons, head-to-head results and historical records. The Engineering Lab displays the underlying cleaning reports, SQL results, quality checks, Delta history and benchmark evidence. The player page demonstrates spatial visualization using generated movement points. Gaussian kernel density estimation converts those points into a smooth heatmap. We clearly distinguish these demonstrations from observed player tracking. The league standings also use simplified rules rather than every official deduction and qualification exception.

Slide 10 - Results and next steps (6:35-7:15)

The saved execution evidence records 10,781 historical matches, 30,186 goals and 30 passing tests. PostgreSQL and MongoDB reconcile to the expected counts. Analytical answers agree across engines, Delta history remains readable, and the event pipeline handles duplicates and invalid payloads. The main improvements would be official competition rules, observed player-event data and a controlled process for promoting staged events into the historical warehouse. For deployment beyond this local teaching environment, we would also add proper authentication, managed secrets, monitoring and tested backups.

Slide 11 - Thank you (7:15-7:25)

Thank you. We're happy to answer questions about the architecture, implementation or results, and demonstrate the working platform.

Timing buffer (7:25-7:30)

Keep these five seconds free.